Capitan — Design Document

App name: Capitan (kept in one config constant so it can be renamed) · **Platform:** iOS via Expo Go · **Status:** Draft v3 · **Author:** Stanley

---

1. Overview

Capitan is an iPhone app for climbing gyms. The climber takes one photo of a wall, and the app identifies the holds, groups them into routes by color, and shows the optimal hand-and-foot sequence up whichever route the climber picks, personalized to their height and wingspan.

Every photo is tagged with the gym (place) it was taken at and added to a growing training corpus. Each time a user asks for a path, the backend runs the model as trained on the current corpus, so results get better as more scans come in. Users can also correct mistakes in hold detection and route colors, and those corrections become labeled training data. The goal is a model that generalizes to any wall, so old scans stay useful even after a gym resets its routes. All of this matters because the dataset will be small at launch.

2. Phasing and ground rules

Work is split into two phases.

Phase 1 — Frontend (Expo Go, iOS). The full app experience, built against a mocked API that has the same shape as the future backend. Anywhere the backend would do work (quality check, nearby-gym matching, hold detection, path planning, auth), the frontend shows a processing state and returns fixture data.

Phase 2 — Backend + ML. Image quality check, place matching, hold detection, route segmentation, personalized path planning, storage, real auth, and the training-corpus loop.

> **Hard gate:** Do not start any Phase 2 work until Stanley explicitly approves it. Finishing Milestone 6 means stopping and asking.

Other rules for whoever builds this (human or Claude Code):

- Everything in Phase 1 must run in **Expo Go on iOS**, with no custom native modules or dev-client-only libraries.
- The app name lives in a single constant (`src/config/app.ts → APP_NAME = "Capitan"`) and in `app.json` . No hardcoded "Capitan" strings anywhere else, so it can be renamed in one place.
- Use a **placeholder logo** (simple monogram or generic mountain mark in the theme colors) stored as a single asset at `assets/logo.png` , so it can be swapped later.
- Anything not specified in this doc goes to Stanley as a question rather than an assumption. Section 11 lists what's still open.
- All UI work uses the **UI UX Pro Max** skill (Section 4).

3. Frontend tech stack

Concern	Choice	Notes
Framework	Expo (latest SDK) + TypeScript	Must run in Expo Go on iOS
Navigation	Expo Router	File-based routes, tab + stack
Camera	`expo-camera`	Single-photo capture
Location	`expo-location`	Foreground location for nearby-gym suggestions
Image prep	`expo-image-manipulator`	Resize/compress before upload
Overlays	`react-native-svg`	Hold outlines, route highlights, hand/foot path markers
Zoom/pan	`react-native-gesture-handler` + `react-native-reanimated`	Pinch-zoom on the wall photo with overlays locked to it
State	Zustand	Lightweight global store
Local persistence	`@react-native-async-storage/async-storage`	Mock accounts, profile, saved scans in Phase 1
API layer	`src/api/` with a `mock` and (later) `http` implementation	Swapped by one config flag

4. Design system (UI UX Pro Max skill)

Install the skill in the project and generate a persisted design system before building any screens:

```
npx ui-ux-pro-max-cli init --ai claude
python3 .claude/skills/ui-ux-pro-max/scripts/search.py \
  "rock climbing gym outdoor fitness mobile app earthy" \
  --design-system --persist -p "Capitan"
python3 .claude/skills/ui-ux-pro-max/scripts/search.py \
  "camera capture image overlay zoom" --stack react-native
```

This produces `design-system/capitan/MASTER.md`. Page-specific overrides (e.g. the camera screen, the path viewer) go in `design-system/capitan/pages/`. Every screen build starts by reading MASTER.md plus any page override, and every milestone ends with the skill's pre-delivery checklist.

Direction: earthy / outdoor climbing. Warm neutrals (sandstone, granite grey, chalk white, deep forest or moss accents), natural textures used sparingly, sturdy readable type. Follow iOS conventions (safe areas, native-feeling navigation, SF-style tap targets of at least 44pt).

Constraint the skill won't know about: gym holds come in arbitrary saturated colors (neon green, hot pink, purple, etc.), and the route overlays have to stay readable on top of them. The UI chrome should therefore stay muted so overlays pop, and overlay styling needs to be tested against real gym photos, not just mockups.

Accessibility: route identity must never rely on color alone, since many climbers are color-blind. Every

route gets a text label ("Blue," "Route 3") and a swatch. Path steps get numbers, and hands vs feet are distinguished by shape and label (e.g. circle "LH 3" vs square "RF 4"), not just color.

5. User flows

First launch: Onboarding (3–4 cards on what the app does and how to take a good photo) → Sign up / log in (mocked) → Climber profile setup (height, wingspan) → Home.

Core scan flow:

1. Tap *Scan wall* → camera screen with framing guide and tips.
2. Take photo → preview (*Retake* / *Use photo*).
3. Pick the place: the app asks for location permission and shows nearby gyms first, with a search field and *Add new place* below. If permission is denied, it falls back to search only.
4. Quality check: frontend sends the image to `checkImageQuality`. If it fails (blurry, too dark, no wall found), show the reason and a *Retake* button. In Phase 1 the mock always passes, with a dev toggle to force a failure.
5. Processing screen ("Analyzing wall…" placeholder).
6. Results: the photo with all detected holds outlined, and a route picker listing each color route with its community grade (if any).
7. Optional: tap *Fix detection* to enter correction mode (see below).
8. Pick a route → path view: that route's holds highlighted, others dimmed, numbered steps showing which hand or foot goes on which hold, computed for the user's height and wingspan.
9. Optional: user enters or confirms a V-scale grade for the route.
10. Scan is saved to the user's history under that place, and the image is added to the training corpus.

Correction mode: from the results screen, the user can reassign a hold to a different route color, delete something that isn't a hold, or add a missed hold by drawing a box around it. Edits show immediately on the photo, and *Done* submits them. Routes and paths are then refetched so the corrected holds are used.

Reopening a saved scan calls the path endpoint again rather than showing a cached result, so the user gets the current model's answer (and their current profile measurements).

History: Saved walls tab → list of places → scans at that place (newest first) → tap to reopen.

6. Screens

Screen	Purpose	Key states
Onboarding	Explain app + photo tips	Skippable; shown once
Auth (sign up / log in)	Mocked account access	Loading, error, validation
Climber profile setup	Height + wingspan entry	Unit toggle (ft/in ↔ cm), validation, skippable with a warning that paths won't be personalized

Home	Entry point: *Scan wall*, recent scans	Empty state for new users
Camera	Capture single photo	Permission denied, framing guide, tips toggle
Photo preview	Confirm or retake	—
Place picker	Tag scan with gym	Nearby list, location denied → search only, no results → add new
Quality check / retake	Backend verdict on photo	Passed, failed with reason, network error
Processing	Wait for analysis	Placeholder progress, cancel
Results / route picker	All holds + list of routes	No routes found, low-confidence warning
Path view	Personalized hand/foot sequence on chosen route	Step-through mode, zoom/pan, recomputing state
Correction mode	Fix hold detection and route colors	Select hold → change color / delete; draw box to add; undo; submit
Grade entry	User enters/confirms the route's V-scale grade	Picker from VB, V0–V17; shows community grade and number of votes
Saved walls	Scans grouped by place	Empty state
Place detail	All scans at one place	—
Profile	Account info, height/wingspan, units, sign out	Editing measurements triggers path recompute next time

7. Frontend ↔ backend API contract

The mock API implements exactly this interface so Phase 2 only swaps the implementation.

```
// Types
type Point = { x: number; y: number };          // normalized 0–1 image coords
type Limb = "LH" | "RH" | "LF" | "RF";
type ClimberProfile = { heightCm: number; wingspanCm: number };
type Hold = {
  id: string;
  polygon: Point[];                              // outline for overlay
  color: string;                                 // route color label, e.g. "blue"
  confidence: number;                            // 0–1
};
type PathStep = { order: number; holdId: string; limb: Limb };
type Route = {
  id: string;
  color: string;
  displayName: string;                           // "Blue", "Route 3"
```

```
  holdIds: string[];
  communityGrade?: { grade: VGrade; votes: number };
};
type Place = { id: string; name: string; distanceM?: number; scanCount: number };
type Scan = {
  id: string; placeId: string; imageUri: string; createdAt: string;
  status: "processing" | "done" | "failed";
  holds: Hold[]; routes: Route[];
};
type HoldCorrection =
  | { kind: "recolor"; holdId: string; newColor: string }
  | { kind: "delete"; holdId: string }
  | { kind: "add"; box: { topLeft: Point; bottomRight: Point }; color: string };
type VGrade = "VB" | `V${0|1|2|3|4|5|6|7|8|9|10|11|12|13|14|15|16|17}`;
type QualityResult =
  | { ok: true }
  | { ok: false; reason: "blurry" | "too_dark" | "no_wall_detected"; message: string };

// Functions (mock now, HTTP later)
checkImageQuality(image): Promise<QualityResult>
createScan(image, placeId): Promise<{ scanId: string }>   // also adds image to training cor
getScan(scanId): Promise<Scan>                             // poll until status === "done"
getOptimalPath(scanId, routeId, profile: ClimberProfile): Promise<PathStep[]>  // never cach
submitGrade(routeId, grade: VGrade): Promise<Route>
submitCorrections(scanId, corrections: HoldCorrection[]): Promise<Scan>  // returns updated
getNearbyPlaces(lat, lng): Promise<Place[]>                // backend matches + dedupes
searchPlaces(query): Promise<Place[]>
createPlace(name, lat?, lng?): Promise<Place>
listScansByPlace(placeId): Promise<Scan[]>
signUp / logIn / logOut / getCurrentUser                   // mocked in Phase 1
updateProfile(profile: ClimberProfile): Promise<void>
```

Mock behavior: realistic delays (1–3 s); 2–3 fixture wall photos with hand-labeled holds and routes; a small fixture list of Bay Area gyms for `getNearbyPlaces`; mock `getOptimalPath` returns slightly different sequences for different heights so the personalization UI can be tested; mock `submitCorrections` applies the edits to the fixture scan so the UI updates realistically; and a dev menu toggle to force quality-check failure, processing failure, and "no routes found."

8. Data model (shared by both phases)

`User` → has one `ClimberProfile`, many `Scan`, many `GradeVote`. `Place` → has many `Scan` and stores coordinates when known. `Scan` → has one image, many `Hold`, many `Route`, and is added to the training corpus on creation. `Scan` also has many `Correction`s (who made it, what changed, when), which are kept as training labels. `Route` → references `Hold`s and has many `GradeVote`s (V-scale only). Optimal paths are not stored as permanent data; they're computed per request from the scan, route, profile, and current model.

9. Phase 1 milestones

M0 — Project setup and design system Expo + TypeScript + Expo Router project runs in Expo Go on an iPhone; APP_NAME constant and placeholder logo in place; UI UX Pro Max installed and MASTER.md

iPhone, `APP_NAME` constant and placeholder logo in place, UI UX Pro Max installed and MASTER.md generated; theme tokens exported from MASTER.md into `src/theme/`; mock API scaffold with fixture data. *Done when:* app boots in Expo Go showing a themed Capitan placeholder screen, and Stanley has approved the generated design system.

M1 — Navigation shell, onboarding, mock auth, profile setup Tab/stack structure for all screens in Section 6 (placeholders OK); onboarding carousel with photo tips; mocked sign up / log in; climber profile setup with height and wingspan and a unit toggle. *Done when:* every screen is reachable, onboarding and login persist across restarts, and measurements are stored in cm regardless of display unit.

M2 — Capture, place picker, quality check Camera with permission handling and framing guide; preview with retake; place picker with location permission, nearby-gym list from the mock, search fallback, and add-new; quality check call with pass and fail/retake paths; image compressed before "upload." *Done when:* pass, forced-fail, and location-denied paths all work end to end on an iPhone.

M3 — Processing and results Processing placeholder with polling; results screen rendering hold polygons from fixture data over the photo, with pinch-zoom/pan keeping overlays aligned; route picker listing routes with text label, swatch, and community grade. *Done when:* overlays stay aligned at every zoom level on a small and a large iPhone.

M4 — Personalized path view, corrections, and grade entry Selected route highlighted, others dimmed; numbered hand/foot markers distinguishable by shape and label; step-through mode; path fetched with the user's profile on every open; correction mode (recolor, delete, draw-to-add, undo, submit); V-scale grade entry. *Done when:* changing height in the profile and reopening a route shows the mock's different sequence; a correction made on the results screen is reflected in the route list and path; and a climber can follow the steps on a fixture wall without explanation.

M5 — Profile and saved walls Profile screen with editable measurements and sign out; auto-save of scans; saved walls grouped by place; place detail list; reopening a saved scan re-requests the path. *Done when:* scans survive app restart and group correctly under places.

M6 — Polish and QA Empty/error/loading states everywhere; accessibility pass (VoiceOver labels, contrast, Dynamic Type, color-independent route and limb identification); UI UX Pro Max pre-delivery checklist; testing on at least two iPhone sizes via Expo Go. *Done when:* checklist passes and Stanley signs off.

> **STOP here. Ask Stanley before starting Phase 2.**

10. Phase 2 outline (draft — not to be started without approval)

Included only so the frontend contract fits what's coming; details will be settled with Stanley before any work begins.

- **M7 Image quality check:** blur/exposure scoring plus an "is this a climbing wall" check, behind `checkImageQuality`.
- **M8 Place matching:** `getNearbyPlaces` and `createPlace` dedupe gyms using coordinates plus fuzzy name matching, so "Mission Cliffs" and "mission cliffs SF" resolve to one place.
- **M9 Hold detection and segmentation:** instance segmentation model producing hold polygons, starting from public climbing-hold datasets and training on the growing corpus.

- **M10 Route grouping by color:** color classification per hold, robust to gym lighting, chalk, and similar-looking colors.
- **M11 Personalized path planning:** hand/foot sequence from start to top, with reach constraints derived from height and wingspan.
- **M12 Training-corpus loop:** every scan and every user correction is added to the corpus, with corrections serving as ground-truth labels. The model is trained to generalize across walls rather than memorize specific ones, so scans from before a reset stay in the training set. The model is retrained or updated as the corpus grows, and every path request uses the current model.
- **M13 API service, storage, real auth:** endpoints from Section 7, image storage, database, and a real auth provider replacing the mock.
- **M14 Integration:** swap the mock API for the real one; end-to-end testing at a real gym.

11. Decisions and open questions

Decided (v2):

- Paths show hands and feet, personalized by height and wingspan.
- Grades are entered by users.
- GPS suggests nearby gyms. The frontend reads location and shows suggestions; matching and deduplication happen in the backend (mocked in Phase 1).
- Auth is mocked in Phase 1.
- Scans add data to the training corpus; each path request uses the model trained on the current corpus.
- iOS only for now.
- App name: Capitan (renameable); placeholder logo.
- Users can correct hold detection and route colors; corrections become training labels.
- Grades use the V-scale only (VB, V0–V17).
- Old scans stay in the training set; the model should generalize to any wall, so there is no per-wall tagging and no "wall" field in the place picker.

Still open: none right now. New questions will be added here as they come up during building.